Conv_Simple — Bench Demo Playbook

From the PLC workbench to a live "text your factory, it tells you what's wrong" demo

---

Audience: You, standing at the bench with the Micro 820 + GS10 rig | **Goal:** get MIRA detecting a real fault on real hardware, today
PLC: Allen-Bradley Micro 820 (2080-LC20-20QBB) @ `192.168.1.100:502` (Modbus TCP) | **PC:** `192.168.1.50` (Ethernet) / Tailscale `100.72.2.99`
Drive: AutomationDirect GS10 DURApulse, Modbus RTU slave node 1, 9600/8N2 | **Sensor just wired:** PE-101 photo-eye → embedded `DI 5`
Slave map: `plc/MbSrvConf_v4.xml` (now 23 coils — `_IO_EM_DI_05` at coil `000023`) | **Engine:** `plc/conv_simple_anomaly/` (A0–A10 live; A2/A7/A12 dormant)
Companion deep-dive: `docs/instructions/Conv_Simple_Complete.html` (RS-485 wiring + Modbus sequencer + UDFB)

> 📖 **What's in here**
>
> 1. **The demo scenario** — the story you're trying to tell, and why it sells.
> 2. **How the system works** — the data path from a beam-break to a phone reply.
> 3. **Where we are right now** — what's done, and the one thing gating everything.
> 4. **The bench playbook** — Step 0 → Step 3, each with what to do, why, and what you should see.
> 5. **The demo script** — exactly what to do and say in front of an audience.
> 6. **Troubleshooting + cheat sheet** — IPs, commands, fault table.

1. The demo scenario — the story you're selling

"Factory technicians spend up to 40% of their time just diagnosing what's wrong. What if a tech could text the machine — in plain English — and the machine told them the fault, the likely cause, and the next thing to check?"

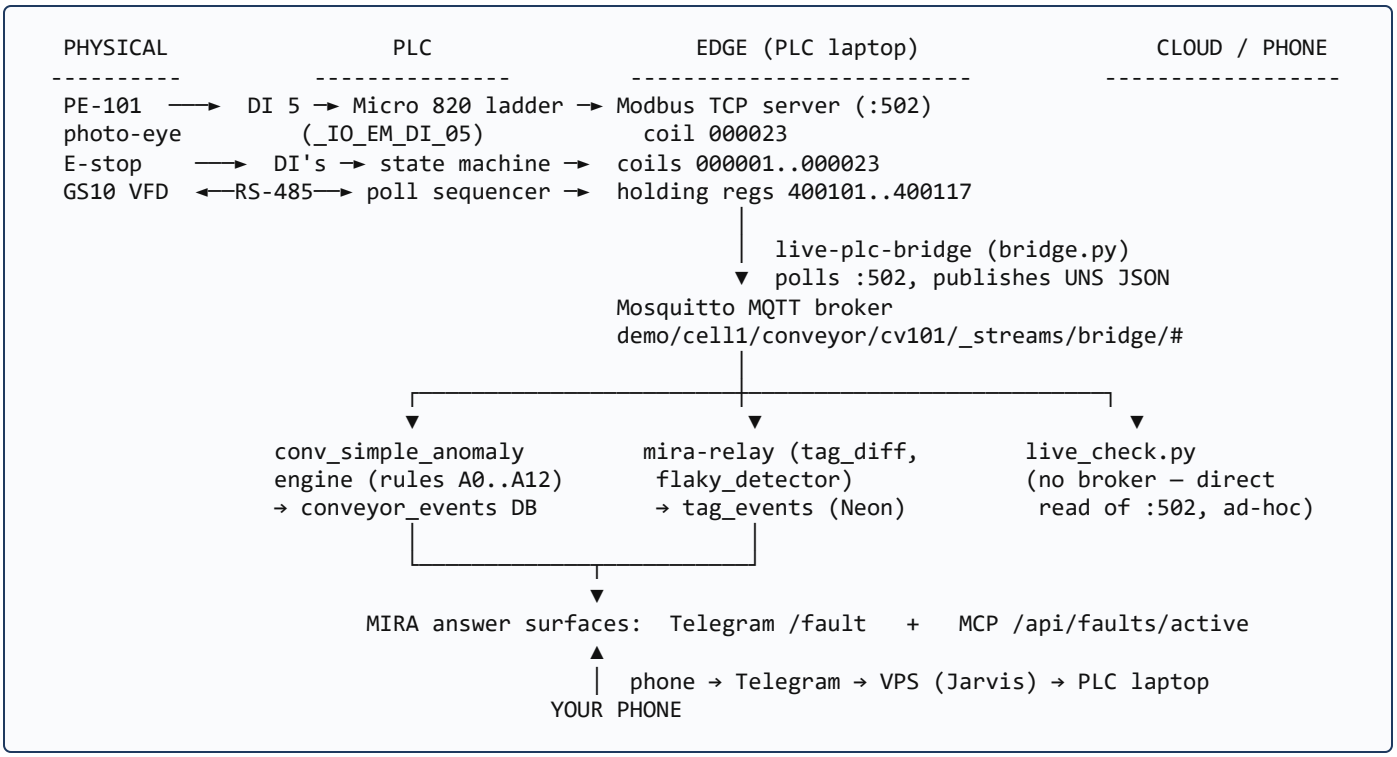
That is MIRA. The demo proves three things in under two minutes, on **real hardware**, not a slideshow:

1. **Real PLC, real fault.** A real Micro 820 + GS10 conveyor rig develops a real problem — a flaky sensor wire, an E-stop, a lost drive link.
2. **The AI diagnoses it.** MIRA reads the live tag stream, recognizes the fault pattern, and explains it the way a senior tech would — fault → likely cause → next check.
3. **From your phone.** The whole thing rides phone → Telegram → cloud → factory floor → back to your phone. No HMI, no manual.

Why this matters: A dashboard tells a manager a number is red. MIRA tells a *technician on the floor* what to do about it. That is the wedge — conversational diagnosis for the person holding the wrench, not another SCADA screen.

2. How the system works — beam-break to phone reply

Every fault follows the same path. Knowing it tells you exactly where to look when something doesn't light up.



The two read paths

`live_check.py` is your **bench multimeter** — it reads the PLC directly and runs the rules, no broker or Docker needed. The `bridge → relay → tag_events` path is the **production pipeline** that feeds Telegram/MCP. Get `live_check.py` green first; it isolates "is the PLC publishing the tag?" from "is the cloud plumbing wired?"

3. Where we are right now

Piece	Status
Modbus TCP to the PLC (`:502`)	✓ **LIVE** — bridge reads it; heartbeat toggling, scan running
Anomaly engine (rules A0–A10)	✓ **MERGED + bench-verified** (fired A1 when the drive link dropped)
Slave-map v2 — PE-101 photo-eye on DI 5	**Code done** (PR #1905) — *needs a controller download to take effect*
Rules A2 / A7 / A12 (VFD fault + photo-eye jam)	Dormant — wake after the reflash publishes their registers
GS10 RS-485 drive link (motor run)	✗ **Not confirmed** — gated by the CCW download below

The one thing gating everything: a clean CCW download.
Historically a `TCPIPObject` download failure aborted the *entire* config transfer — both the Modbus slave map and the RS-485 serial config. That single failure is why the photo-eye never reflashed *and* the drive never talked. A fix was written (static IP `192.168.1.100`, replacing the APIPA `169.254.x` that broke the TCP object) but **never verified**. Confirming a clean download is today's highest-value move — it can unblock both payoffs at once.

4. The bench playbook

0 Master gate — get ONE clean CCW download UNBLOCKS EVERYTHING

Do this first. Both Step 1 and Step 2 ride on it.

1. Stage the updated map into the CCW project so it goes down with this build: `python plc/deploy_modbus_map.py`
2. Confirm the PC is on `192.168.1.50` and you can reach the PLC: `ping 192.168.1.100`.
3. In CCW: **Build**, then **Download to controller**. Let it run to 100% with no error dialog.
4. ☐ **If the download fails** with the TCP/"out of sync" error → unplug Ethernet for the download and use the **USB cable** instead. USB bypasses the `TCPIPObject` path entirely — this is the untried tactic.
5. After it completes: **Serial Port** → **Diagnose** must report **"in sync."**

✓**You should see:** the download reaches 100% with zero failures, and Serial Port → Diagnose says "in sync."

Why this matters: "In sync" means the controller's embedded serial config finally matches the project. That is the precondition the drive link has been waiting on for months — and the same download carries your new 23-coil slave map.

Report back here
Tell me: did it download clean, and does Diagnose say "in sync"? Your answer decides whether we sprint to the easy demo (Step 1) or also chase the motor (Step 2).

1 Easy demo win — the live photo-eye feed NO MOTOR NEEDED

The moment Step 0 succeeds, your slave-map v2 is live. This path needs no VFD, so it's your fastest route to a demoable moment.

1. Read the PLC directly and run the rules for a few seconds:
   `python plc/conv_simple_anomaly/live_check.py --secs 4`
2. **Break the PE-101 beam** with your hand a few times while it runs.
3. Watch the decoded tag `plc/di/di05_photoeye` flip `True/False` with the beam.

✓**You should see:** `plc/di/di05_photoeye` toggles in lock-step with your hand breaking the beam.

Why this matters: This proves the full sensor → DI 5 → coil 000023 → Modbus → decode chain end to end. Once the beam reads live, the flaky-input detector (which keys on a rapidly toggling sensor) has real data to fire on — that is the "flaky sensor wire" story, the most relatable fault on a factory floor.

> **If di05_photoeye never appears**
> The read block `(22,1)` covers coil 000023. If it errors, the download didn't carry the new map — re-do Step 0 (the map edit is only real after a successful download).

2 Full demo — VFD link → first motor run STRETCH GOAL

Only chase this once Step 0's "in sync" holds — that's the gate the drive link was always waiting on.

1. Read every VFD tag and check comm health:
   `python plc/vfd_diag.py`
2. Expect `vfd_comm_ok = TRUE` and `ErrorID ≠ 255`. `255` means the MSG never completed a real transaction (UART silent).
3. If still timing out after "in sync": swap the **D+/D-** pair once, then toggle GS10 `P09.04` between `12` and `13` (8N1 ↔ 8N2). Re-run `vfd_diag.py` after each change.
4. On healthy comm: command **FWD run** → first motor turn → then **REV**.

✓**You should see:** `vfd_comm_ok=TRUE`, a non-255 ErrorID, and the motor physically turns on a FWD command.

> **Don't rewrite the program**
> The ladder/ST is correct. If comm fails, it is sync or physical (wiring/baud), never the code. The deep RS-485 reference is `Conv_Simple_Complete.html`.

3 Close the loop to MIRA THE PAYOFF

This turns a green `live_check` into an actual phone reply. The cloud-side prep (seed `approved_tags`, relay env) can be staged in parallel while you're on the hardware.

1. Start the bridge on the PLC laptop (only host routed to the PLC subnet) so it publishes the UNS stream.

2. With `approved_tags` seeded + `RELAY_TAG_EVENTS=1`, the relay logs tag changes to `tag_events`; the flaky detector raises a `flaky_signal_alert`.
3. From your phone, message the MIRA Telegram bot: "*What's wrong with the conveyor?*" → it answers from the live fault.

✓**You should see:** breaking the beam ~7× quickly produces a flaky-sensor alert that MIRA reports in Telegram / on `/api/faults/active`.

5. The demo script — what to do and say

Rehearse this exact sequence. Total run time ~2 minutes.

#	You do	You say	Audience sees
1	Hold up phone, conveyor running on the bench	"This is a real conveyor — Allen-Bradley PLC, real drive. Watch me ask it what's wrong, like a tech would."	A live machine, a phone
2	Wiggle / rapidly break the PE-101 beam (simulating a loose sensor wire)	"A sensor wire just started acting up — the kind of intermittent fault that eats a tech's afternoon."	You touching the rig
3	Text the bot: "what's wrong with the conveyor?"	"I'm not reading a manual or a dashboard. I'm just texting the factory."	A normal text message
4	Show the reply	"It caught the flaky sensor, told me the likely cause, and the next thing to check — in plain English."	MIRA's diagnosis on the phone
5	(If Step 2 is live) command a run / inject a drive fault	"And it's not just sensors — it reads the drive, the safety circuit, the whole machine."	Motor runs / fault caught

Fallbacks (always have one ready)

- **If Step 2 (motor) isn't live:** the photo-eye flaky-sensor story (Steps 1+3) is a complete demo on its own. Lead with it.
- **If the cloud loop hiccups:** run `live_check.py` on the laptop screen — the rules firing on a real beam-break is still a compelling, honest proof.
- **If the PLC laptop drops:** have a screen recording of a clean run as the ultimate backstop.

6. Troubleshooting + cheat sheet

Symptom	Likely cause	Do this
CCW download fails / "out of sync"	TCPIPObject over Ethernet	Download over USB cable instead; re-check Diagnose
`di05_photoeye` missing in live_check	New map not actually downloaded	Re-run Step 0; map edits are inert until a clean download
`live_check` coil read error @22	Coil 000023 not mapped on controller	Confirm `deploy_modbus_map.py` ran *before* the download
`vfd_comm_ok=FALSE` , ErrorID 255	Serial not in sync / UART silent	Confirm "in sync"; swap D+/D-; toggle P09.04 12↔13
Beam toggles but no Telegram reply	Cloud loop (bridge/relay/seed)	Verify bridge running + `approved_tags` seeded + `RELAY_TAG_EVENTS=1`

ⓘ One-page reference

Addresses

PLC (Modbus TCP)	`192.168.1.100 : 502`, unit 1
PLC laptop	`192.168.1.50` (Ethernet) · `100.72.2.99` (Tailscale)
UNS prefix	`demo/cell1/conveyor/cv101`
Photo-eye coil	`000023` = `_IO_EM_DI_05` = pymodbus offset 22 → topic `plc/di/di05_photoeye`

Commands (run on the PLC laptop)

Stage map into CCW	`python plc/deploy_modbus_map.py`
Live rules vs real PLC	`python plc/conv_simple_anomaly/live_check.py --secs 4`
VFD comm health	`python plc/vfd_diag.py`
Unit tests (sanity)	`python -m pytest plc/conv_simple_anomaly/` (26)

The priority call

Closest-to-market-ready today = **Step 0** → **Step 1** → **Step 3** (live AI-detected flaky sensor answered on a phone). The motor run (Step 2) is the stretch goal once the download is clean — don't let it block the demo.

Generated for the Conv_Simple bench session. Companion docs: `plc/RESUME_1668_PLC_FEED.md` (full feed spec) · `docs/instructions/Conv_Simple_Complete.html` (RS-485 / Modbus / UDFB deep-dive).